SatsPath Engine v0 — Walkthrough

What Engine v0 is

SatsPath Engine v0 is an experimental signed payment resolver and router. It is **not** a production wallet or automatic payment engine.

It can: - Resolve a local or peer-registered signed profile - Select a payment rail (Lightning, on-chain, Ark) based on live mempool fees - Fetch a real LNURL invoice and display a scannable QR code - Display a BIP-21 URI with a QR code for on-chain payments - Preview which swap directive would be needed (testnet only, no execution)

It cannot (and intentionally does not): - Move funds automatically - Sign Bitcoin transactions - Broadcast anything to the network - Store or generate seed phrases - Execute mainnet swaps

Swap engine status — Engine v0 scaffolding only

The `satspath-swaps` crate is an **experimental testnet-only scaffold** for Boltz Exchange v2 swap integration.

Claim and refund transaction construction is not implemented (Engine v1 work).
Mainnet payment execution is not implemented.

- PSBT signing is not implemented
- Mainnet execution paths are closed
- The swap engine is only reachable via `--experimental-swaps --testnet`
- Without those flags, no swap code runs at all

Default behavior — what `satspath pay` does

```
satspath pay rodrigodiazgt7@gmail.com 1000
```

1. Resolves the alias to a signed local profile
2. Verifies the identity signature (ECDSA/secp256k1)
3. Fetches live mempool fee rates from mempool.space
4. Selects the best payment rail based on fees and amount
5. For Lightning: performs LNURL-pay two-step fetch → real BOLT11 invoice → QR
6. For on-chain: builds BIP-21 URI → QR

SatsPath does not send any funds. No keys are touched. No transactions are signed or broadcast. The displayed invoice/URI is for the user to scan with their own wallet.

Experimental swap engine — testnet intent preview only

```
satspath pay rodrigodiazgt7@gmail.com 1000 --experimental-swaps --testnet
```

1. Same resolution and routing
2. Shows swap directive intent only
3. Does NOT execute the swap
4. Does NOT build any transaction
5. `--experimental-swaps` without `--testnet` is rejected with a hard error

No funds are moved. This is a preview of what a swap would look like.

Persistence and secrets

- All local state lives in `.satspath/` (gitignored — never committed)
- `LocalPeerRegistry` stores SHA-256(canonical_identifier) as the DB key — raw email is never stored as primary key

- `SwapStore` writes sensitive swap secrets (`preimage_hex`, `refund_key_hex`, `claim_key_hex`) only when an AES-256-GCM encryption key is provided
- Writing sensitive swap material to plaintext storage is rejected at the record level
- No private keys, seeds, macaroons, or API tokens are committed to this repository

ARK bridge status

The `ark-bridge/` directory contains a JSON-RPC bridge skeleton that would connect the Rust CLI to the ARK client-side validation SDK.

Current status: - The TypeScript bridge compiles and handles all protocol methods with stub responses - VTXO DAG validation is not implemented (requires the full Ark SDK — tracked as Engine v1) - The Rust `ArkBridge::spawn()` call is non-fatal: if the bridge is unavailable, the CLI continues in pointer/intent mode and prints a clear warning - Ark payments in Engine v0 display the payment pointer and an explicit experimental warning

What is implemented vs. what is not

Feature	Status
Signed profile resolution (local registry)	
secp256k1 identity signature verification	
Live mempool fee fetch (mempool.space)	
Lightning rail selection (amount $<$ 100k sats)	
On-chain rail (fastestFee 20 sat/vB = next block)	
Ark fallback (high fees)	
LNURL-pay two-step invoice fetch	
BOLT11 invoice amount verification (HRP parse)	
Terminal QR code (Dense1x2 unicode)	
BIP-21 on-chain URI with QR	
LocalPeerRegistry (SHA-256 keyed, no raw email)	
SwapStore AES-256-GCM encryption	
SwapStore sensitive-record guard (plaintext rejected)	
Boltz API client (testnet scaffolding)	scaffold
Submarine/Reverse/Chain swap creation scaffolding	scaffold
Claim/Refund transaction construction	Engine v1
PSBT signing	Engine v1
BOLT11 expiry verification (needs bech32 data decode)	Engine v1
Ark VTXO DAG validation	Engine v1
Mainnet swap execution	Intentionally closed

Engine v1 scope (future work)

- PSBT construction and signing (rust-bitcoin + BDK)
- BOLT11 expiry parsing via bech32 data field decode
- Ark VTXO DAG validation via full ARK SDK
- Cooperative Taproot/MuSig2 spend for chain swaps
- BIP-353 DNS-based payment address resolution

Wallet Integration & Post-Quantum Security (Fase 2 & 3)

The Arkade Money Wallet has been fully integrated with the SatsPath protocol in the frontend via WASM.

- **Post-Quantum Cryptography (PQC):** The protocol now uses a hybrid signature scheme (`ML-DSA-65-Schnorr`) for generating and verifying identity keys. The `pqc_seed` is encrypted and stored locally via the Web Crypto API alongside the classical identity key.
- **SSRF Protection:** Resolvers (both WASM and Core) now strictly validate URLs and block loopback, private, and internal metadata IP ranges to prevent malicious profile endpoints from exploiting the client's internal network.
- **Nostr Profiles:** The wallet now successfully builds, signs, and publishes NIP-78 SatsPath profiles to Nostr relays.
- **Routing & Execution:** The Wallet's Send Flow uses the SatsPath resolver WASM module to dynamically quote and fallback between Lightning, On-chain, and Ark endpoints seamlessly.